

SQLx:

The Rust SQL Toolkit



An async, pure Rust SQL crate features compile-time checked queries without a domain specific language. This article will give a basic whirlwind tour of the library by showing CRUD operations and the transactions supported using it.

SQLx is a Rust crate for interacting with your database. Some of its features that distinguish it from other crates in the Rust ecosystem are:

- Built with async IO in mind
- Compile-time checked queries
- Database agnostic
- Postgres/MySQL drivers are written in pure Rust (as opposed to them being a FFI binding to a C library)
- Works with various runtimes

Database setup

For our demo, we will start with a simple database with two tables and a simple relationship between them:

```
CREATE TABLE "person"("id" SERIAL8 PRIMARY KEY UNIQUE,"name"
TEXT NOT NULL,"age" smallint NOT NULL,"email" TEXT NULL)
CREATE TABLE "book"("id" SERIAL8 PRIMARY KEY UNIQUE,"name"
TEXT NOT NULL,"isbn" TEXT NOT NULL,"person" INT8 NOT NULL)
ALTER TABLE "book" ADD CONSTRAINT "book_person_fkey" FOREIGN
KEY("person") REFERENCES "person"("id") ON DELETE RESTRICT
ON UPDATE RESTRICT
```

We have two tables:

- Person
- Book

And there is a relationship between the 'book' and the 'person' tables. Make sure you have a local (or remote, if you prefer) PostgreSQL running to test it out. You will also need the `sqlx cli` in your `$PATH`.

Rust project setup

We will use Cargo to create a new project. Cargo is the package manager for Rust. It can download your Rust

package's dependencies, compile your packages, make distributable packages, and upload them to `crates.io`. You can refer to Rust documentation for more details.

```
❏ cargo new sqlx-demo
```

Created binary (application) 'sqlx-demo' package

The project is set up as a binary project:

```
❏ tree
```

```
├── Cargo.toml
└── src
    └── main.rs
```

1 directory, 2 files

Let's export the database credentials in an environment variable:

```
export DATABASE_URL=postgres://postgres:postgres@localhost/
sqlx-demo
```

Next, we will add the SQL file (`20211223133946_initial_setup.sql`) as part of the migration:

```
❏ sqlx migrate add initial_setup
```

Creating migrations/20211223133946_initial_setup.sql

Congratulations on creating your first migration!

Did you know you can embed your migrations in your application binary? On startup, after creating your database connection or pool, add: